

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE

Semester Genap 2026/2027

MODUL 11- React Routing & Multi-Page App + Integrasi API

Nama Mahasiswa : Aisyah Ayu Salsabila
NIM : 23051430001
Kelas : A2_Teknik Industri
Tanggal Praktikum : 1 Mei 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 10 (Pertemuan 10)
Topik Utama	React Routing & Multi-Page App + Integrasi API
Sub-Topik	Single Page Application (SPA) Navigation, React Router DOM, dan Fetch API dalam React.
Durasi Praktikum	340 Menit
Tanggal Praktikum	1 Mei 2026
Nama Mahasiswa	Aisyah Ayu Salsabila
NIM	23051430001
Kelas	A2_Teknik Industri

B. TUJUAN PEMBELAJARAN

1	Memahami konsep SPA (Single Page Application).
2	Mampu mengatur navigasi antar halaman tanpa reload browser menggunakan `react router-dom`.
3	Menerapkan `fetch` API di dalam `useEffect` untuk mengambil data eksternal.
4	Membuat struktur aplikasi multi-halaman (Dashboard, Inventori, Settings).

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Ekstensi Live Server		Perubahan web realtime
4	Laptop	Windows 11	Prasarana Praktikum
5	Modul Aplikasi Web dan Mobile	2025/2026	Manual Praktikum
6	Console browser dan ekstensi Quokka.js		JavaScript alat pemroses data

7.	Internet		Mengakses API publik
8.	Node.js		Untuk npm
9.	Project React		Hasil Pertemuan 10

D. DASAR TEORI

1. Latar Belakang Kebutuhan Routing dalam Sistem Industri

Dalam lingkungan industri modern, aplikasi web tidak lagi cukup hanya menampilkan satu halaman statis. Sebuah sistem monitoring pabrik membutuhkan beberapa modul terpisah, seperti:

- Dashboard (ringkasan KPI)
- Inventori (stok bahan baku)
- Laporan Kualitas (data cacat produksi)

Pada aplikasi web tradisional (multi-page), setiap pergantian menu menyebabkan *browser reload* seluruh halaman. Hal ini mengakibatkan:

- Flicker (kedipan) yang mengganggu pengalaman pengguna.
- Kehilangan state sementara (misalnya data filter yang sudah diisi).
- Waktu ulang yang tidak efisien, terutama di jaringan pabrik yang terbatas.

Single Page Application (SPA) memuat seluruh kerangka aplikasi di awal, kemudian pergantian halaman dilakukan secara dinamis di sisi klien tanpa *reload* browser. Hal ini membuat aplikasi terasa seperti *native app* (cepat dan mulus).

2. Konsep Dasar Routing dengan React Router DOM

react-router-dom adalah pustaka standar untuk mengatur navigasi di aplikasi React. Komponen utamanya meliputi:

Komponen	Fungsi	Analogi Industri
<BrowserRouter>	Membungkus seluruh aplikasi agar mampu menangani URL	Seperti panel kontrol pusat yang mengatur semua mesin
<Routes>	Tempat mendaftarkan semua rute yang tersedia	Daftar stasiun kerja di pabrik
<Route>	Memetakan URL ke komponen tertentu	Peta jalur: dashboard : Komponen Dashboard
<Link>	Navigasi antar halaman tanpa reload	Tombol pindah stasiun tanpa mematikan mesin
<NavLink>	Seperti Link tapi bisa tahu sedang aktif	Indikator lampu yang menyala di menu aktif

Perbedaan penting:

- `<a href="#"...">` → menyebabkan *full page reload* (TIDAK digunakan dalam SPA).
- `<Link to="#"...">` → navigasi SPA tanpa reload (WAJIB digunakan).

3. Pengambilan Data Eksternal dengan Fetch API + useEffect

Di halaman Inventori, data stok tidak ditulis keras (*hardcoded*) melainkan diambil dari server eksternal (dalam praktikum ini menggunakan *mock API* jsonplaceholder.typicode.com).

useEffect adalah Hook yang menjalankan kode *side effect* (seperti mengambil data) setelah komponen selesai dirender.

Siklus pengambilan data yang benar:

1. Komponen Inventori pertama kali dimuat (mount).
2. useEffect dengan dependency (kosong) dijalankan SATU KALI.
3. Di dalamnya, fetch() mengirim permintaan ke API.
4. Saat data tiba, setProducts(data) menyimpan ke state.
5. React otomatis me-render ulang tabel dengan data yang sudah masuk.

4. Siklus Navigasi dalam SPA (Tanpa Reload)

Berikut diagram alur navigasi saat pengguna berpindah dari Dashboard ke Inventori:

1. Pengguna mengklik `<Link to="/inventori">`.
2. react-router-dom mencegah klik tersebut.
3. Browser tidak mengirim permintaan baru ke server.
4. Router mencocokkan URL `/inventori` dengan daftar `<Route>`.
5. Komponen `<Inventori />` dirender menggantikan Dashboard.
6. State aplikasi tetap terjaga. Jika ada data di Dashboard, tidak hilang.

D.1 Konsep Utama

1. SPA memungkinkan navigasi antar halaman (Dashboard, Inventori, Settings) tanpa reload browser.
2. React Router DOM menyediakan BrowserRouter, Routes, Route, dan Link untuk pemetaan URL dan navigasi tanpa kehilangan state.
3. Fetch API dalam useEffect([]) mengambil data eksternal sekali saat komponen pertama kali dimuat.

4. Conditional Rendering menampilkan pesan "Memuat data..." saat loading, serta menangani halaman 404 dengan path="*".

D.2 Keterkaitan dengan Teknik Industri

Penguasaan SPA dan routing memungkinkan mahasiswa Teknik Industri membangun sistem monitoring pabrik yang modular, seperti dashboard KPI, inventori bahan baku, dan laporan kualitas cacat produksi dalam satu aplikasi tanpa jeda reload. Dengan kemampuan fetch data dari backend, mahasiswa dapat mengintegrasikan data sensor atau database produksi secara real-time ke antarmuka web. Teknik navigasi mulus ini juga mendukung pengambilan keputusan cepat di lantai produksi karena operator tidak perlu menunggu halaman memuat ulang saat berpindah modul. Kemampuan ini menjadi fondasi pengembangan alat bantu digital interaktif untuk monitoring, pelacakan stok, dan visualisasi data industri secara efisien.

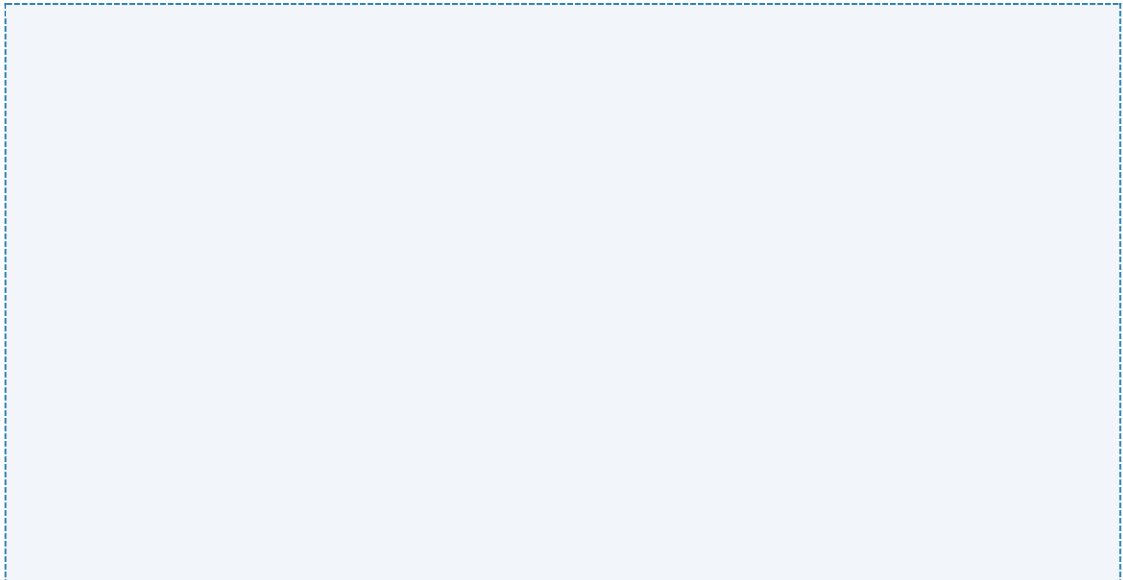
E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Setup Router

Buka file utama `src/main.jsx` (atau `index.jsx` tergantung versi Vite). Bungkus aplikasi dengan `BrowserRouter`

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App.jsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
)
```



Gambar E.1 – Setup Router

E.2 Langkah 2 – Membuat Halaman (Pages)

Pengisian src halaman dengan jsx,

Untuk Dashboard.jsx

```
import React from 'react';
import { Link } from 'react-router-dom';
function Dashboard() {
  return (
    <div className="p-5">
      <h1>Dashboard Utama Pabrik</h1>
      <p>Selamat datang di sistem monitoring terpadu.</p>
      <Link to="/inventori" className="btn btn-primary">
        Ke Halaman Inventori
      </Link>
    </div>
  );
}
export default Dashboard;
```

Untuk Inventori.jsx

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';

function Inventori() {
  const [products, setProducts] = useState([]);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts')
```

```

        .then(res => res.json())
        .then(data => {
            setProducts(data.slice(0, 5));
        })
        .catch(err => console.log(err));
    }, []);

    return (
        <div className="container mt-4">
            <h1>Data Inventori Bahan Baku</h1>
            <Link to="/" className="btn btn-secondary mb-3">Kembali ke
Dashboard</Link>
            <table className="table table-striped">
                <thead>
                    <tr>
                        <th>ID Item</th>
                        <th>Nama Bahan</th>
                        <th>Status Supplier</th>
                    </tr>
                </thead>
                <tbody>
                    {products.map((item) => (
                        <tr key={item.id}>
                            <td>{item.id}</td>
                            <td>{item.title}</td>
                            <td><span className="badge bg-success">Available</span></td>
                        </tr>
                    ))}
                </tbody>
            </table>
        </div>
    );
}

export default Inventori;

```

Untuk NotFound.jsx

```

import React from 'react';
import { Link } from 'react-router-dom';

function NotFound() {
    return (
        <div className="text-center mt-5">
            <h1 className="display-1">404</h1>
            <p>Halaman tidak ditemukan dalam sistem pabrik.</p>

```

```
        <Link to="/" className="btn btn-dark">Kembali ke Home</Link>
      </div>
    );
  }

  export default NotFound;
```

Gambar E.2 – Halaman

E.3 Langkah 3 – Mengatur Route di App.jsx

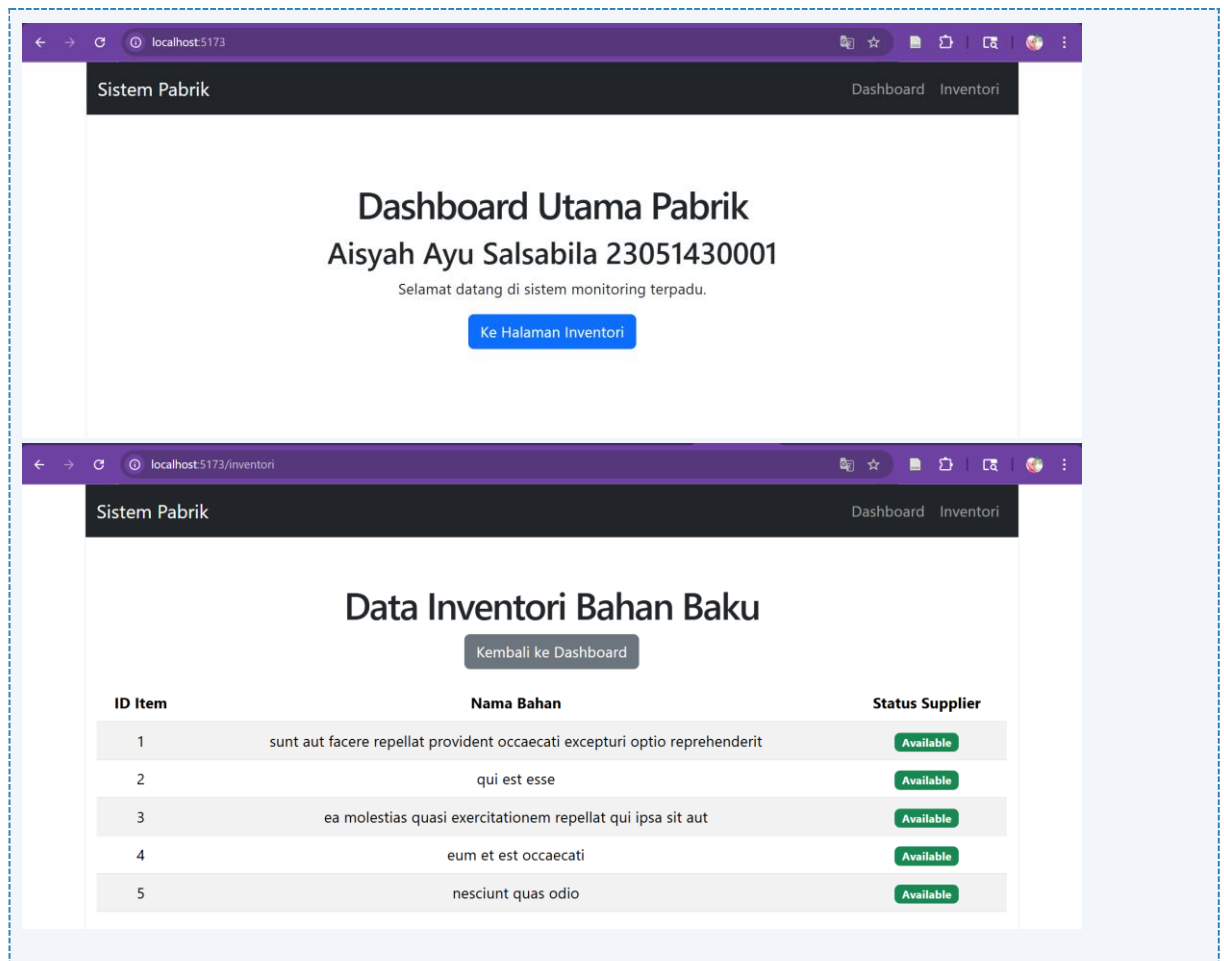
Sekarang kita tentukan kapan komponen mana yang muncul berdasarkan URL.

```
import React from 'react';
import { Link } from 'react-router-dom';

function Navbar() {
  return (
    <nav className="navbar navbar-expand-lg navbar-dark bg-dark mb-4">
      <div className="container">
        <Link className="navbar-brand" to="/">Sistem Pabrik</Link>
        <div className="navbar-nav">
          <Link className="nav-link" to="/">Dashboard</Link>
          <Link className="nav-link" to="/inventori">Inventori</Link>
        </div>
      </div>
    </nav>
  );
}

export default Navbar;
;
```

Screenshot Hasil:



Gambar E.3 – Data Inventory dari Api Dummy

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – (Dynamic Route)

Pertanyaan / Instruksi Latihan:

Latihan 1(Dynamic Route) *Tambahkan Route untuk detail item. Misal path 'inventori/:id'. Di halaman Inventori, ubah 'Nama Bahan' menjadi Link ke halaman detail.*

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';

function Inventori() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts')
```

```

        .then(res => res.json())
        .then(data => {
            setProducts(data.slice(0, 5));
            setLoading(false);
        })
        .catch(err => console.log(err));
    }, []);

    if (loading) {
        return <div className="text-center mt-5">Memuat data...</div>;
    }

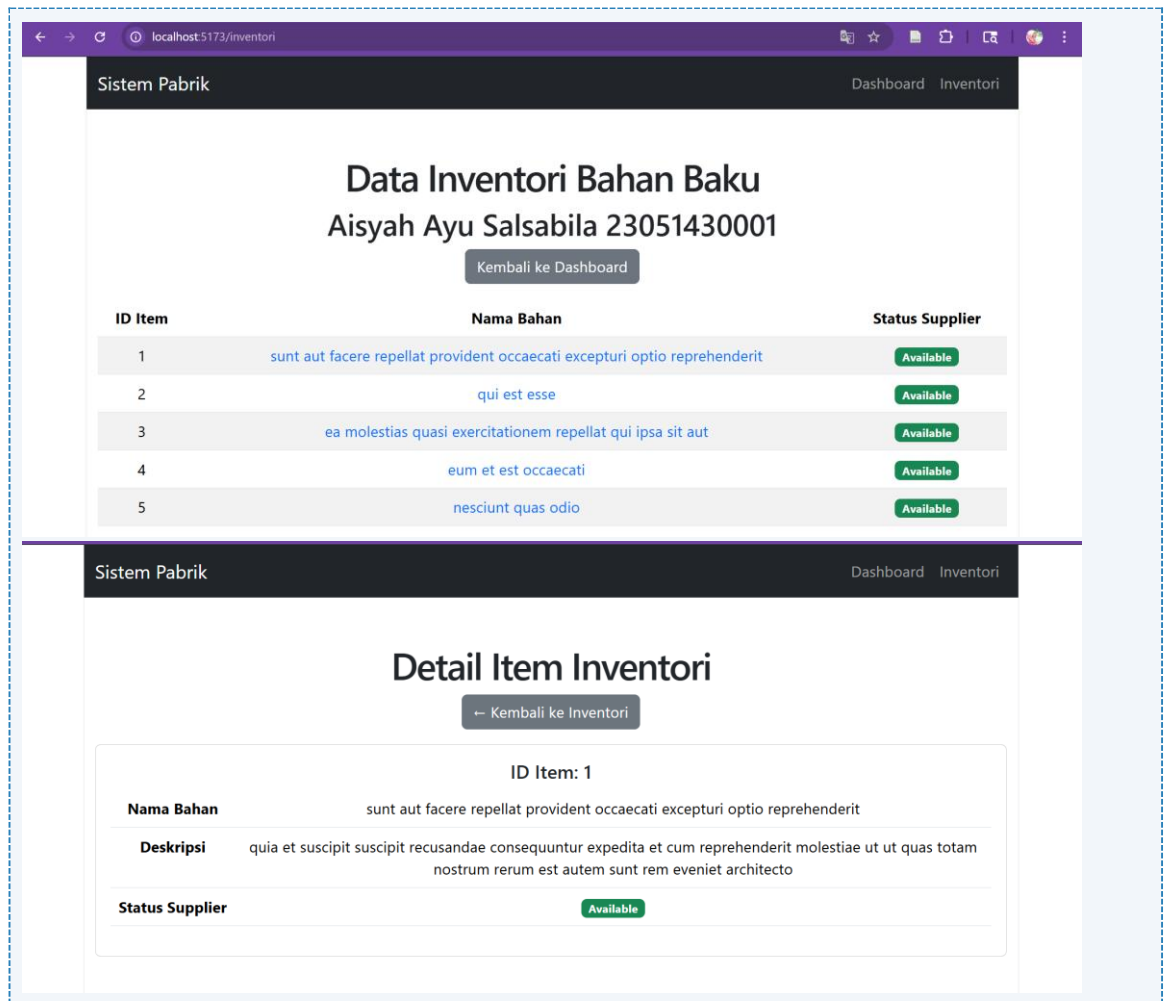
    return (
        <div className="container mt-4">
            <h1>Data Inventori Bahan Baku</h1>
            <h2>Aisyah Ayu Salsabila 23051430001</h2>
            <Link to="/" className="btn btn-secondary mb-3">Kembali ke
Dashboard</Link>
            <table className="table table-striped">
                <thead>
                    <tr>
                        <th>ID Item</th>
                        <th>Nama Bahan</th>
                        <th>Status Supplier</th>
                    </tr>
                </thead>
                <tbody>
                    {products.map((item) => (
                        <tr key={item.id}>
                            <td>{item.id}</td>
                            <td>
                                <Link to={`/inventori/${item.id}`} className="text-
decoration-none">
                                    {item.title}
                                </Link>
                            </td>
                            <td><span className="badge bg-success">Available</span></td>
                        </tr>
                    ))}
                </tbody>
            </table>
        </div>
    );
}

export default Inventori;

```

app jsx nya ditambahkan :

```
import DetailItem from './Halaman/DetailItem';
dan
<Route path="/inventori/:id" element={<DetailItem />} />
```



Gambar F.1 – Dependency Array useEffect

F.2 Latihan 2 – Loading State

Pertanyaan / Instruksi Latihan:

Latihan 2 (Loading State): Di halaman `Inventori.jsx`, tambahkan state `loading` (boolean). Set `loading = true` sebelum fetch, dan `false` setelah data masuk. Tampilkan teks "Memuat data..." jika loading true.

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from 'react';
```

```
import { Link } from 'react-router-dom';

function Inventori() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    setLoading(true);

    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(res => res.json())
      .then(data => {
        setProducts(data.slice(0, 5));
        setLoading(false);
      })
      .catch(err => {
        console.log(err);
        setLoading(false);
      });
  }, []);

  if (loading) {
    return (
      <div className="text-center mt-5">
        <h3>Memuat data...</h3>
        <div className="spinner-border text-primary mt-3" role="status">
          <span className="visually-hidden">Loading...</span>
        </div>
      </div>
    );
  }

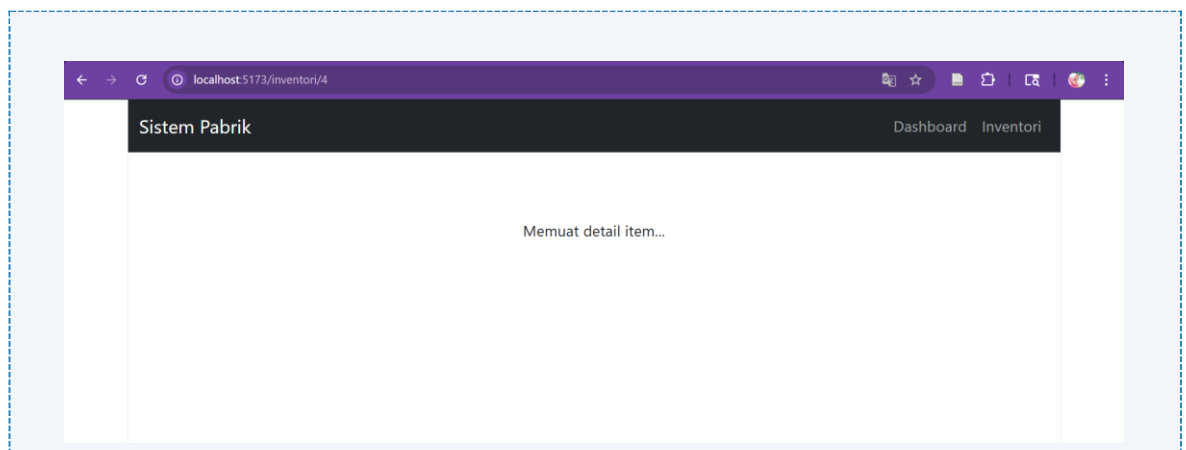
  return (
    <div className="container mt-4">
      <h1>Data Inventori Bahan Baku</h1>
      <Link to="/" className="btn btn-secondary mb-3">Kembali ke Dashboard</Link>
      <table className="table table-striped">
        <thead>
          <tr>
            <th>ID Item</th>
            <th>Nama Bahan</th>
            <th>Status Supplier</th>
          </tr>
        </thead>
        <tbody>
```

```

        {products.map((item) => (
          <tr key={item.id}>
            <td>{item.id}</td>
            <td>
              <Link to={`/${inventori}/${item.id}`} className="text-decoration-none">
                {item.title}
              </Link>
            </td>
            <td><span className="badge bg-success">Available</span></td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
);
}

export default Inventori;

```



Gambar F.2 – Loading State

F.3 Tugas Proyek Mini – (Kartu Karyawan)

Deskripsi proyek mini:

Tugas Proyek Mini (Kalkulator OEE Sederhana): a. Buat komponen form input: `Plan Time`, `Run Time`, `Total Parts`, `Good Parts`. b. Gunakan State untuk menyimpan nilai-nilai ini. c. Hitung OEE secara otomatis (Real-time) setiap kali input berubah. d. Rumus OEE = (Availability x Performance x Quality). e. Tampilkan hasil OEE dalam persen. Jika OEE < 50%, warnanya merah. Jika > 85%, warnanya hijau.

Penjelasan pendekatan/solusi yang digunakan:

Membuat komponen baru yaiku kalkulator oee, state sebagai penyimpanan nilai , OEE terhitung otomatis dengan input , menghasilkan persenn dan tanda warna.

Jawaban untuk app.jsx:

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';

function LaporanKualitas() {
  const [loading, setLoading] = useState(true);
  const [cacatData, setCacatData] = useState([]);

  useEffect(() => {
    const mockCacat = [
      { id: 1, tanggal: "2026-05-05", mesin: "Mesin Krill", jenisCacat: "Putus benang", jumlah: 12, shift: "Pagi" },
      { id: 2, tanggal: "2026-05-05", mesin: "Mesin Jahit", jenisCacat: "Berlubang terlewat", jumlah: 5, shift: "Siang" },
      { id: 3, tanggal: "2026-05-05", mesin: "Mesin Obras", jenisCacat: "Kain terbuka", jumlah: 8, shift: "Pagi" },
      { id: 4, tanggal: "2026-05-05", mesin: "Mesin Krill", jenisCacat: "Putus Benang", jumlah: 3, shift: "Malam" },
      { id: 5, tanggal: "2026-05-06", mesin: "Mesin Jahit Kecil", jenisCacat: "Jahit terbuka terlewat", jumlah: 7, shift: "Pagi" },
      { id: 6, tanggal: "2026-05-06", mesin: "Mesin Obras", jenisCacat: "Benang mudah putus", jumlah: 4, shift: "Siang" },
      { id: 7, tanggal: "2026-05-07", mesin: "Mesin Krill", jenisCacat: "Tidak rapat", jumlah: 6, shift: "Pagi" },
      { id: 8, tanggal: "2026-05-07", mesin: "Mesin Pembuat Kancing", jenisCacat: "Penyok", jumlah: 9, shift: "Malam" },
    ];

    // Simulasi loading
    setTimeout(() => {
      setCacatData(mockCacat);
      setLoading(false);
    }, 500);
  }, []);

  // Hitung total cacat
  const totalCacat = cacatData.reduce((sum, item) => sum + item.jumlah, 0);

  // Hitung cacat per jenis
  const cacatPerJenis = {};
  cacatData.forEach(item => {
```

```

    if (cacatPerJenis[item.jenisCacat]) {
      cacatPerJenis[item.jenisCacat] += item.jumlah;
    } else {
      cacatPerJenis[item.jenisCacat] = item.jumlah;
    }
  });

  if (loading) {
    return (
      <div className="text-center mt-5">
        <h3>Memuat data laporan kualitas...</h3>
        <div className="spinner-border text-danger mt-3" role="status">
          <span className="visually-hidden">Loading...</span>
        </div>
      </div>
    );
  }

  return (
    <div className="container mt-4">
      <h1 className="mb-4">Laporan Kualitas Produksi</h1>
      <p className="text-muted mb-4">Data cacat produksi periode 1-4
Mei 2026</p>

      </* Ringkasan Card */>
      <div className="row mb-4">
        <div className="col-md-4">
          <div className="card text-white bg-danger">
            <div className="card-body">
              <h5 className="card-title">Total Cacat</h5>
              <h2 className="display-4">{totalCacat}</h2>
              <p className="card-text">Unit</p>
            </div>
          </div>
        </div>
        <div className="col-md-4">
          <div className="card text-white bg-warning">
            <div className="card-body">
              <h5 className="card-title">Jenis Cacat</h5>
              <h2 className="display-
4">{Object.keys(cacatPerJenis).length}</h2>
              <p className="card-text">Kategori</p>
            </div>
          </div>
        </div>
      </div>
      <div className="col-md-4">

```

```

        <div className="card text-white bg-info">
          <div className="card-body">
            <h5 className="card-title">Hari Produksi</h5>
            <h2 className="display-4">4</h2>
            <p className="card-text">Hari</p>
          </div>
        </div>
      </div>
    </div>
  </div>

  { /* Tabel Data Cacat */ }
  <div className="card">
    <div className="card-header bg-dark text-white">
      <h5 className="mb-0">Detail Cacat per Produksi</h5>
    </div>
    <div className="card-body">
      <table className="table table-striped table-bordered">
        <thead className="table-danger">
          <tr>
            <th>ID</th>
            <th>Tanggal</th>
            <th>Mesin</th>
            <th>Jenis Cacat</th>
            <th>Jumlah</th>
            <th>Shift</th>
          </tr>
        </thead>
        <tbody>
          {cacatData.map((item) => (
            <tr key={item.id}>
              <td>{item.id}</td>
              <td>{item.tanggal}</td>
              <td>{item.mesin}</td>
              <td>
                <span className="badge bg-
danger">{item.jenisCacat}</span>
              </td>
              <td>
                <span className="badge bg-dark">{item.jumlah}
unit</span>
              </td>
              <td>{item.shift}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </div>

```



```

    </div>
  </div>

  {/* Ringkasan per Jenis Cacat */}
  <div className="card mt-4">
    <div className="card-header bg-secondary text-white">
      <h5 className="mb-0">Ringkasan per Jenis Cacat</h5>
    </div>
    <div className="card-body">
      <table className="table table-sm">
        <thead>
          <tr>
            <th>Jenis Cacat</th>
            <th>Total Unit</th>
            <th>Persentase</th>
          </tr>
        </thead>
        <tbody>
          {Object.entries(cacatPerJenis).map(([jenis, jumlah]) => (
            <tr key={jenis}>
              <td>{jenis}</td>
              <td>{jumlah}</td>
              <td>
                <div className="progress">
                  <div
                    className="progress-bar bg-danger"
                    style={{ width: `${(jumlah / totalCacat) *
100}%` }}
                  >
                    {((jumlah / totalCacat) * 100).toFixed(1)}%
                  </div>
                </div>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </div>

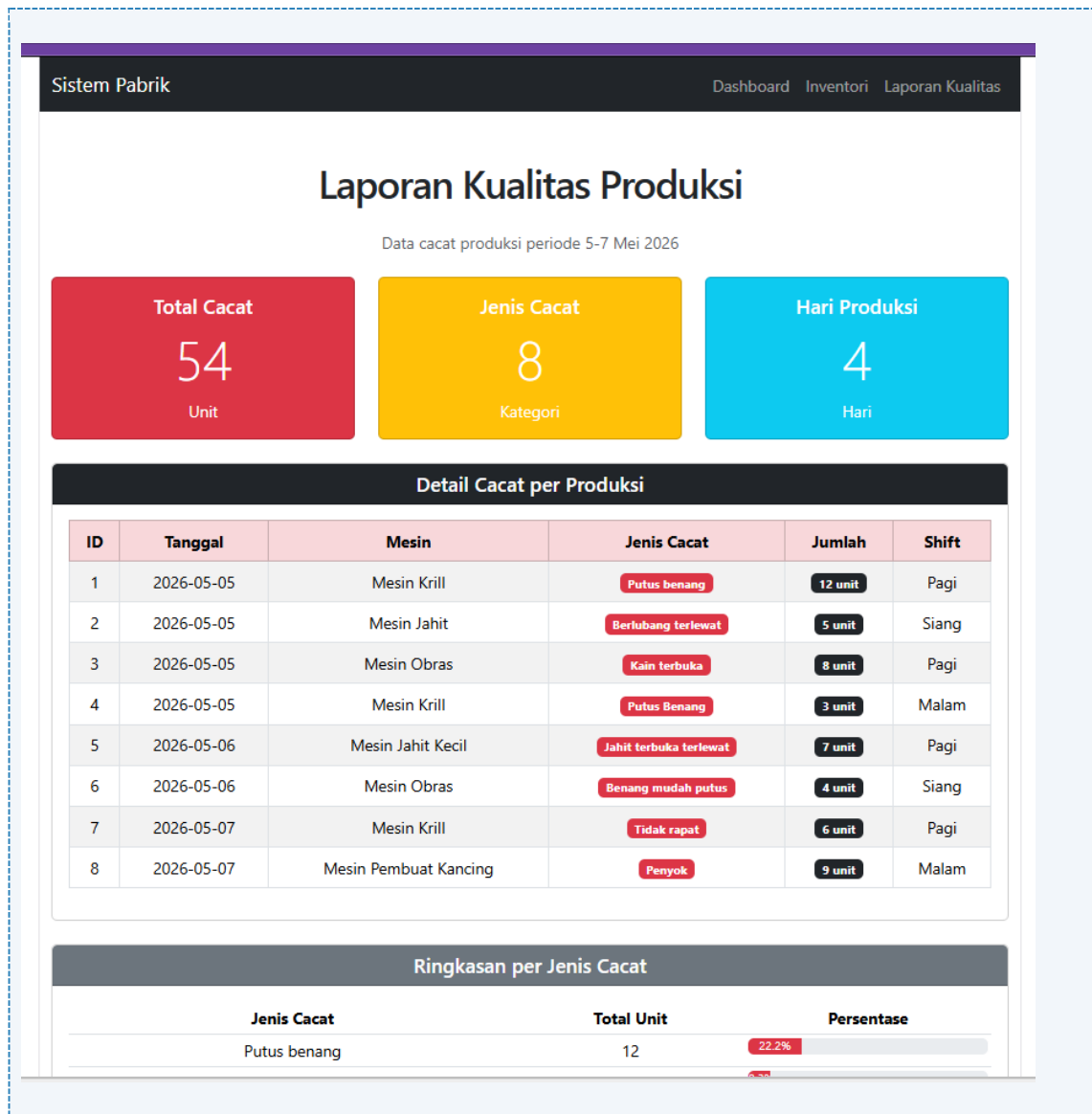
  <div className="mt-3 mb-4">
    <Link to="/" className="btn btn-secondary">← Kembali ke
Dashboard</Link>
  </div>
</div>
);

```

```
}  
  
export default LaporanKualitas;
```

Kode Untuk APP.jsx

```
import React from 'react';  
import { Routes, Route } from 'react-router-dom';  
import Dashboard from './Halaman/Dashboard';  
import Inventori from './Halaman/Inventori';  
import DetailItem from './Halaman/DetailItem';  
import LaporanKualitas from './Halaman/LaporanKualitas';  
import NotFound from './Halaman/NotFound';  
import Navbar from './Komponen/Navbar';  
  
function App() {  
  return (  
    <>  
      <Navbar />  
      <Routes>  
        <Route path="/" element={<Dashboard />} />  
        <Route path="/inventori" element={<Inventori />} />  
        <Route path="/inventori/:id" element={<DetailItem />} />  
        <Route path="/laporan-kualitas" element={<LaporanKualitas />} />  
      </Routes>  
      <Route path="*" element={<NotFound />} />  
    </>  
  );  
}  
  
export default App;
```



Gambar F.3 – Laporan Kualitas Produksi

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum sesuai teori SPA dan React Router DOM: navigasi antar halaman berjalan tanpa reload browser. Fetch API dalam useEffect berhasil mengambil data eksternal satu kali, dengan loading state menampilkan pesan "Memuat data..." saat proses berlangsung. Penambahan halaman modular dan transisi mulus membuktikan aplikasi telah menerapkan konsep SPA dengan benar..

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
--------------------------------	------------------------

Navigasi menyebabkan reload browser karena menggunakan tag <a>	Gunakan komponen <Link> dari react-router-dom agar navigasi tanpa reload
useEffect berulang terus-menerus (infinite loop)	Pastikan dependency array diisi dengan benar; [] kosong agar hanya berjalan sekali
Data tidak muncul setelah fetch, padahal tidak error	Periksa apakah setProducts(data) sudah dipanggil dan state sudah di-render di komponen
Halaman 404 tidak tampil saat mengakses URL salah	Tambahkan route dengan path="" yang mengarah ke komponen NotFound
Parameter URL (misal /inventori/1) tidak terbaca	Gunakan hook useParams() untuk menangkap nilai parameter dari URL

G.3 Kaitan dengan Penerapan di Industri

Penerapan SPA dan routing dalam praktikum ini relevan dengan sistem monitoring industri modern, seperti dashboard pabrik yang memerlukan perpindahan cepat antar modul produksi, inventori, dan laporan kualitas tanpa jeda reload. Teknik navigasi mulus menggunakan React Router DOM memungkinkan operator pabrik mengakses data real-time secara efisien, sementara fetch API dalam useEffect mendukung integrasi dengan database sensor atau backend produksi. Dengan demikian, penguasaan materi ini menjadi fondasi bagi mahasiswa Teknik Industri untuk mengembangkan alat bantu digital interaktif dalam pengambilan keputusan di rantai produksi.

H. KESIMPULAN

1	SPA berhasil diimplementasikan dengan navigasi antar halaman tanpa reload browser menggunakan React Router DOM.
2	Routing dikelola dengan BrowserRouter, Routes, Route, dan Link untuk memetakan URL ke komponen yang sesuai.
3	Fetch API dalam useEffect([]) berhasil mengambil data eksternal satu kali saat komponen pertama kali dimuat.
4	Loading state dengan conditional rendering menampilkan pesan "Memuat data..." selama proses pengambilan data.
5	Halaman 404 dan modul baru (LaporanKualitas) dapat ditambahkan secara modular tanpa mengganggu halaman lain.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
-----	------------------------------------

[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Chambers, J. M. (2014). <i>Object-Oriented Programming , Functional Programming and R</i> . 29(2), 167–180. https://doi.org/10.1214/13-STS452
[3]	Gunawan, R., Suherman, Y., & Wibowo, S. S. (2022). <i>Rancang Bangun Sistem Informasi Verifikasi Dan Validasi Data Pengajuan Tender Berbasis Web Menggunakan Model Terstruktur dan UML Menurut Instruksi Presiden Nomor 3 Tahun mengembangkan</i> . 16(62), 10–19.
[4]	Mulawarman Munsyir, S.E., S.Si., M. K., Eko Prasetyo Widhi, M. K., Eko Wahyudi, M. T., Dr. Bagus Setya Rintyarna, M. K., & Vandika, A. Y. (2024). <i>Belajar Algoritma dan Pemrograman dengan Javascript</i> .
[5]	Kusnawi, Romadhon, W., & Dhuhita, W. M. P. (2024). <i>Menyelami dunia React.JS: Best practice berdasarkan pengalaman di industri IT</i> . Penerbit ANDI.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
	
Aisyah Ayu Salsabila NIM: 23051430001	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

PANDUAN DAN KETENTUAN PENGISIAN LAPORAN

Baca dengan seksama sebelum mengisi template laporan ini

1. Aturan Umum

a.	Laporan dikerjakan secara INDIVIDUAL, kecuali ada instruksi khusus dari dosen.
b.	Isi laporan harus merupakan hasil pekerjaan sendiri. Plagiarisme (menyalin laporan teman secara identik) akan mendapatkan nilai NOL (0) untuk kedua pihak.
c.	Bagian Dasar Teori HARUS ditulis dengan kalimat dan bahasa sendiri, bukan disalin mentah dari modul atau internet.
d.	Screenshot WAJIB memperlihatkan nama/NIM mahasiswa pada salah satu bagian tampilan (bisa di judul halaman web atau bagian lain) sebagai bukti keaslian pekerjaan.
e.	Semua teks dalam kotak yang berwarna abu-abu/biru muda dan bersifat miring adalah instruksi yang HARUS diganti dengan isi yang sesungguhnya.

2. Format File dan Ketentuan Pengumpulan

Hal	Ketentuan
Format File	Microsoft Word (.docx). Konversi ke PDF jika diminta dosen.
Nama File	Laporan_Modul[No]_[NIM]_[NamaLengkap].docx Contoh: Laporan_Modul02_12320001_AhmadDhani.docx
Ukuran Kertas	A4 (210 × 297 mm)
Margin	Atas: 3 cm Bawah: 3 cm Kiri: 4 cm Kanan: 3 cm
Font Teks Isi	Arial, ukuran 11pt, spasi 1.15
Font Kode Program	Courier New, ukuran 10pt, latar gelap (sudah tersedia di template)
Batas Pengumpulan	7 hari kalender setelah tanggal pelaksanaan praktikum
Media Pengumpulan	Github repository mahasiswa + konfirmasi email ke dosen
Subjek Email	[WebLab-No.Modul] NIM - Nama Lengkap Contoh: [WebLab-02] 12320001 - Ahmad Dhani

3. Kompilasi Laporan Akhir Semester

PENTING – LAPORAN AKHIR SEMESTER

Pada akhir semester, seluruh laporan modul (Modul 1–12, kecuali Modul 13 UTS dan Modul 16 UAS) akan dikompilasi menjadi SATU BERKAS LAPORAN AKHIR. Ketentuan kompilasi:

a.	Laporan setiap modul dicetak/disimpan sebagai satu dokumen terpisah sesuai template ini.
b.	Kompilasi disatukan dengan urutan: Halaman Cover Akhir → Daftar Isi → Laporan Modul 1 → Laporan Modul 2 → ... → Laporan Modul 12 → Laporan Modul 14 & 15.
c.	Laporan UTS (Modul 13) dan UAS (Modul 16) dikumpulkan secara TERPISAH dengan ketentuan yang diumumkan saat ujian.
d.	Setiap laporan modul di dalam kompilasi diawali dengan halaman Cover (Bagian A di template ini) sebagai pemisah antar modul.
e.	Nama file laporan akhir: LaporanAkhir_[NIM]_[NamaLengkap].docx
f.	Laporan akhir dikumpulkan paling lambat H-3 sebelum UAS.

4. Ketentuan Pengumpulan via GitHub

```

PraktikumWebMob_TekInd_2026/
├── README.md
├── Laporan/
│   ├── Laporan_Modul01_[NIM].docx
│   ├── Laporan_Modul02_[NIM].docx
│   └── ... (dst hingga Modul 15)
├── Pertemuan01_[NIM]_[Nama]/
│   ├── index.html
│   └── style.css
├── Pertemuan02_[NIM]_[Nama]/
└── ... (dst)

```

Pastikan setiap Commit memiliki pesan yang deskriptif. Contoh: "Menambahkan Laporan Modul 02 – CSS Dasar". Lakukan Push ke GitHub setiap akhir praktikum.

5. Konsekuensi Keterlambatan Pengumpulan

Keterlambatan	Konsekuensi
1–2 hari	Pengurangan nilai 10% dari total skor laporan
3–4 hari	Pengurangan nilai 25% dari total skor laporan
5–7 hari	Pengurangan nilai 50% dari total skor laporan
> 7 hari	Laporan TIDAK DITERIMA (nilai 0) – wajib temui dosen untuk klarifikasi
Tidak dikumpulkan	Nilai laporan = 0, mahasiswa tidak diperkenankan mengikuti UAS

— Selamat mengerjakan —